Estrutura de Dados II

Prof. Vanessa Matias Leite

1

Árvore Geradora Mínima

2

Árvore em grafo

Seja T uma árvore com n vértices, então:

- T é conexo e sem ciclos;
- T possui n – 1 arestas;
- Cada aresta de T é uma ponte;
- T é um grafo planar (sem o cruzamento de arestas);
- Se n>1, então T possui pelo menos dois vértices folhas (ou terminais).

3

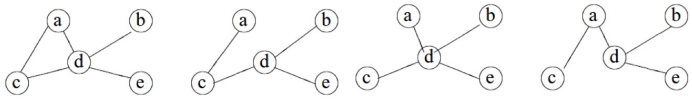
Árvore Geradora

- Alguns problemas de otimização combinatória podem ser formulados através de um tipo de grafo específico conhecido como árvore. Uma árvore é um grafo conexo (existe caminho entre qualquer par de seus nós) e acíclico (sem ciclos). Ou seja, uma árvore é um grafo conexo sem ciclos.
- Dado um grafo conectado **e não orientad**o, uma árvore geradora desse grafo é um subgrafo que é uma árvore e conecta todos os nós juntos.

4

Árvore Geradora

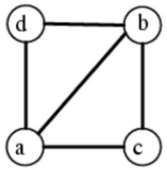
- Um único grafo pode ter muitas árvores diferentes.
- Uma árvore geradora mínima (MST) para um grafo ponderado, conectado e não orientado é uma árvore geradora com peso/custo/distância menor ou igual ao peso/custo/distância de todas as outras árvores geradoras. O custo de uma árvore geradora é a soma dos custos de cada aresta da árvore de arborescência.



5

Árvore Geradora

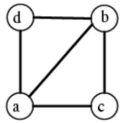
- Liste todas as 8 árvores geradoras do grafo abaixo

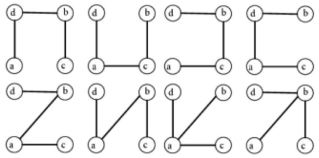


6

Árvore Geradora

- Liste todas as 8 árvores geradoras do grafo abaixo





7

Árvore Geradora Mínima

- Seja G= (V, E) um grafo conexo com uma função de peso w: E → R
- O peso de uma árvore geradora T de G é definido como a soma dos pesos de todas as arestas de

$$w(T) = \sum_{(u,v)\in T} w(\in u, v)$$

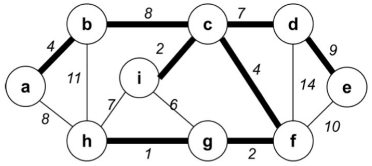
- A meta é achar uma árvore geradora de peso mínimo para G:

  Árvore Geradora Mínima / Minimum Spanning Tree

8

Árvore Geradora Mínima

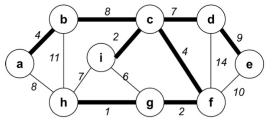
- Só existe uma única árvore geradora mínima para um grafo conexo?

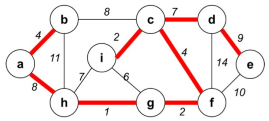


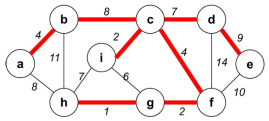
9

Árvore Geradora Mínima

- Só existe uma única árvore geradora mínima para um grafo conexo?



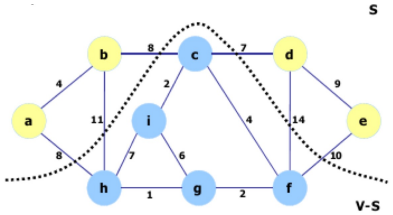




10

Corte

- Corte: Um corte (S, V-S) de um grafo não direcionado G = (V, E) é uma partição de seu conjunto de vértices

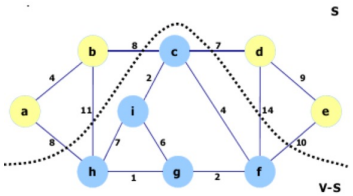


$S = \{a, b, d, e\}$

$V - S = \{h, i, c, g, f\}$

11

Corte



$S = \{a, b, d, e\}$

$V - S = \{h, i, c, g, f\}$

- As arestas (a,h) (b,h), (b,c), (c,d), (d,f), (e,f) **cruzam** o corte
- Dentre elas, a de menor custo (c,d) é dita **aresta leve**
- O corte **respeita** as arestas (a,b), (d,e), (c,f), (f,g), (g,h), (g,i), (c,i), (h,i)

12

Árvore Geradora Mínima

- Existem dois algoritmos clássicos na literatura para resolver o problema da AGM:
  - Algoritmo de Prim;
  - Algoritmo de Kruskal;
- Ambos são considerados **algoritmos gulosos.**
- A estratégia gulosa defende que a menor escolha a cada passo deve ser feita, mesmo que tal escolha não nos leve a uma solução ótima ao final da execução.

MELHOR ESCOLHA IMEDIATA.

13

Algoritmo de Prim

- Começa em um vértice até atar todos os vértices.
- Analisa a cada nó todas as possibilidades de transição e seus custos e escolhe o menor deles.
- Vai analisando a cada iteração o menor caminho e guarda-o na solução que será retornada no final.

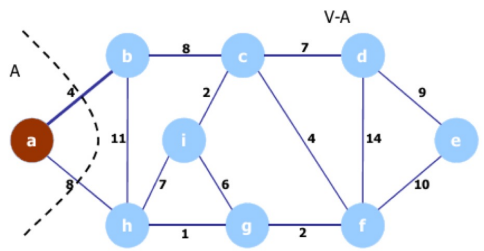
14

Algoritmo de Prim

- Considera **um vértice inicialmente na árvore**
- A cada iteração, o algoritmo procura a aresta de menor peso que conecte um vértice da árvore a outro que ainda não esteja na árvore.
- Esse vértice é adicionado a árvore e o processo se repete.
- Esse processo continua até que
  - Todos os vértices façam parte da árvore
  - Não se pode encontrar uma aresta que satisfaça essa condição (grafo desconexo)

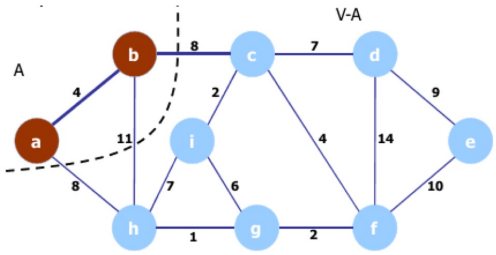
15

Algoritmo de Prim



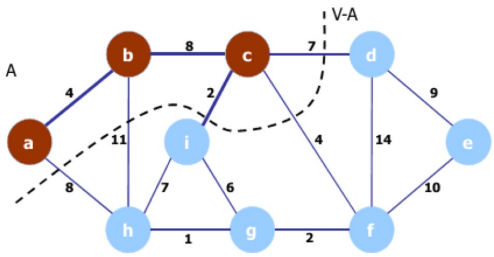
16

Algoritmo de Prim



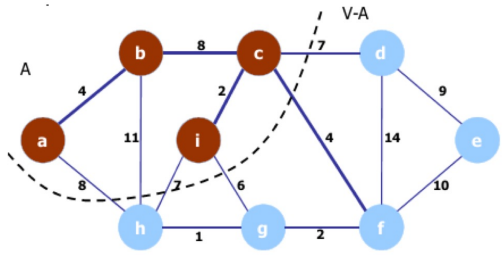
17

Algoritmo de Prim



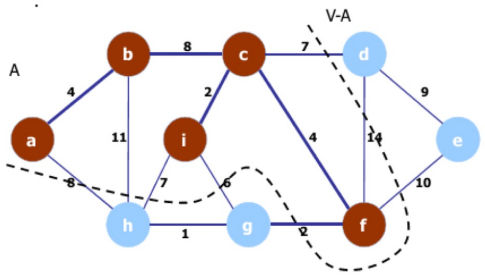
18

Algoritmo de Prim



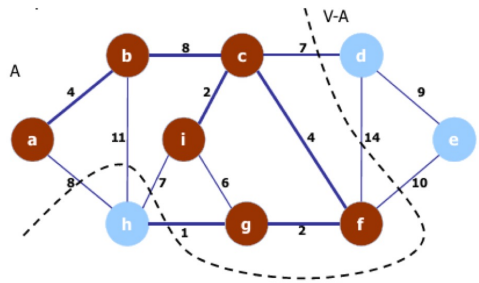
19

Algoritmo de Prim



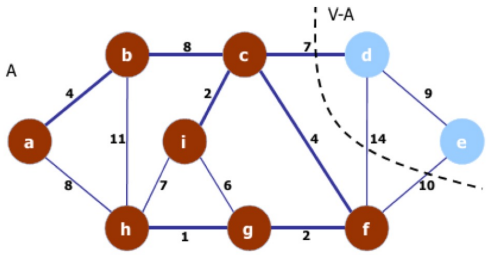
20

Algoritmo de Prim



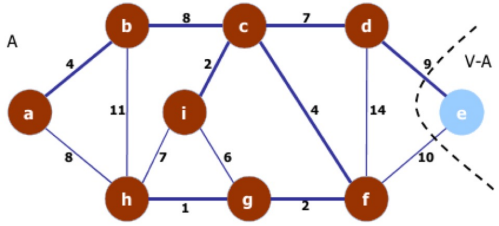
21

Algoritmo de Prim



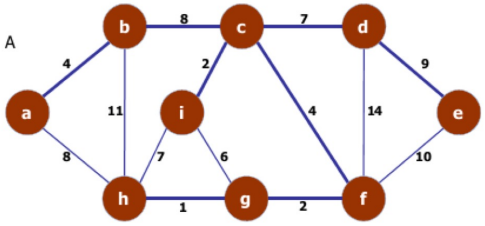
22

Algoritmo de Prim



23

Algoritmo de Prim



24

```
PRIM(G(V, E, w), s): A      //Parâmetros: representação de grafo (V, E, w)
    A ← ∅    //A: conjunto (lista) de arestas da árvore geradora mínima
    para cada vértice v em V faça
        dist [v] ← ∞
        pai[v] ← null
        interligado[v] = false
    fim-para
    dist[s] ← 0
    interligado[s] ← false   //interligado: diz se o vértice já está interligado
    Q ← V – {s}    //Q: conjunto (lista) de vértices a interligar
    para cada vértice v adjacente a s faça
        se dist[v] > w(s, v) então
            dist[v] ← w(s, v)
            pai[v] ← s
        fim-se
    fim-para
    enquanto Q ≠ ∅  faça
        u ← i : min{dist[i], ∀ i ∈ Q}
        Q ← Q – {u}
        interligado [u] ← true
        para cada vértice v adjacente a u faça
            se dist[v] > w(u, v) então
                dist[v] ← w(u, v)
                pai[v] ← u
            fim-se
        fim-para
        A ← A ∪ {(pai[u], u)}  //Adiciona aresta (pai[u], u) à árvore geradora mínima, representada por A
    fim-enquanto
    retorne A
FIM
```

Similar ao algoritmo de Dijkstra!

25

Algoritmo de Kruskal

- O algoritmo de Prim se inicia com um vértice e cresce uma única árvore a partir dele.
- O algoritmo de Kruskal constrói uma floresta (várias árvores) ao longo do tempo, e que são unidas ao final do processo.

26

Algoritmo de Kruskal

- Princípio: a aresta de menor peso sempre pertence à AGM
- Escolha a aresta de menor peso entre todas as arestas que conectam quaisquer dois vértices em A
- A nova aresta não pode ligar vértices na mesma árvore (não pode formar ciclo)

27

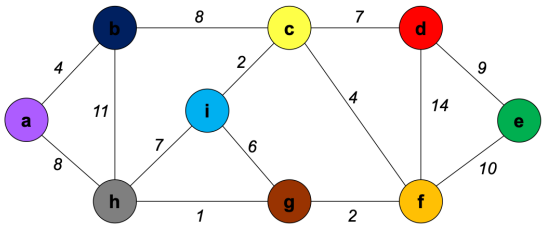
Algoritmo de Kruskal

Funcionamento

- Considera cada vértice como uma árvore independente (floresta)
- A cada iteração, o algoritmo procura a aresta de **menor peso** que conecta duas árvores diferentes
- Os vértices das árvores selecionadas passam a fazer parte de uma mesma árvore
- Esse processo continua até que
  - Todos os vértices façam parte da árvore
  - Não se pode encontrar uma aresta que satisfaça essa condição (grafo desconexo)

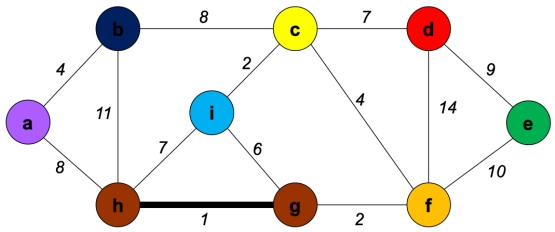
28

Algoritmo de Kruskal



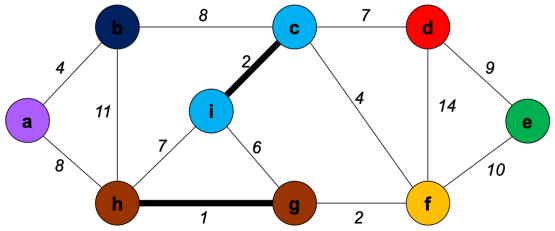
29

Algoritmo de Kruskal



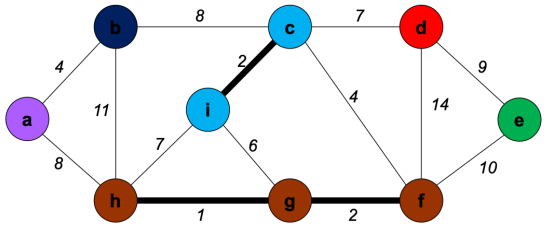
30

Algoritmo de Kruskal



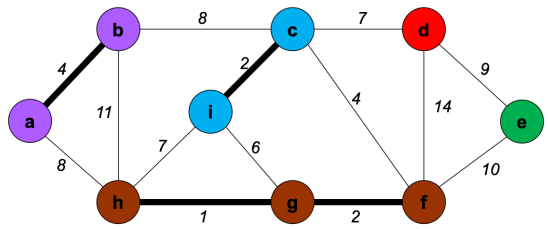
31

Algoritmo de Kruskal



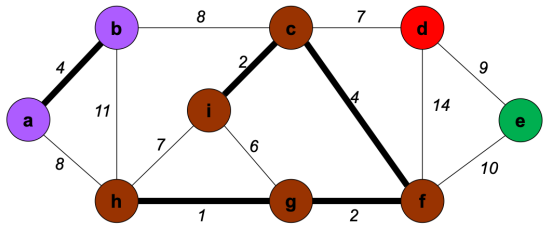
32

Algoritmo de Kruskal



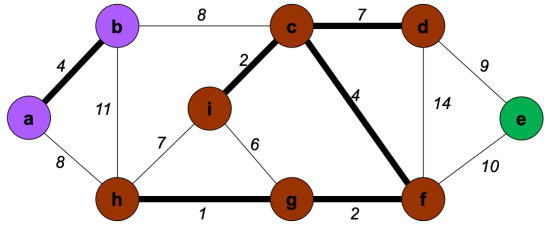
33

Algoritmo de Kruskal



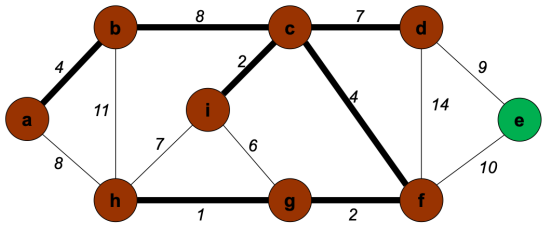
34

Algoritmo de Kruskal



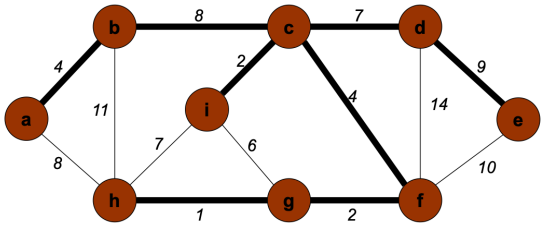
35

Algoritmo de Kruskal



36

Algoritmo de Kruskal



37

Algoritmo de Kruskal

```
KRUSKAL( G(V, E, w)) : A   //Parâmetros: representação de grafo (V, E, w)
    A ← ∅   //A: conjunto (lista) de arestas da árvore geradora mínima (agm)
    C ← ∅   //C: vetor que indica a qual árvore pertence cada vértice
    para i = {1, ..., n} em V faça
        C[i] ← i   //make-set(i): Marca cada vértice pertencente a um conjunto (árvore) distinto
    fim-para
    ordene as arestas de E por peso
    para cada aresta (u, v) em E, em ordem de peso, faça
        se C[u] ≠ C[v]   //find-set[u] ≠ find-set[u]: Vértices u e v pertencem a conjuntos distintos
            A ← A ∪ {(u, v)}   //Adiciona a aresta (u, v) à agm A
            k ← C[u]
            para i = {1, ..., n} em C faça   //union(u, v): Todos os vértices que pertenciam
                se C[i] = k                   ao conjunto C[u] passarão a pertencer
                    C[i] ← C[v]               ao conjunto C[v]
                fim-se
            fim-para
        fim-se
    fim-para
    retorne A
FIM
```

38

Algoritmos	Características
Dijkstra	-Possui um ponto de partida; -Pode ser orientado ou não; -O Custo das arestas não podem ser negativos; -Grafo Conexo
Prim	-Possui um ponto de partida; -Não pode ser orientado; -Os pesos podem ser iguais; -Combinação linear entre os vértices -Grafo Conexo; -Acíclico;
Kruskal	-Não possui um ponto de partida; -Não pode ser orientado; -Acíclico;

39